Labo 10 — Exploitation et bonnes pratiques

Théorie — ce qui se passe après le `up` : observer, limiter, surveiller, durcir, distribuer, nettoyer, et faire redémarrer tout seul. Le labo qui transforme « ça tourne chez moi » en « ça tourne, et on sait pourquoi ».

Objectifs

- Observer un conteneur en production : `logs`, `inspect`, `top`, `stats`, `events`.
- Poser des limites CPU et mémoire, comprendre l'OOM killer et le lien avec la JVM.
- Écrire un `HEALTHCHECK` et savoir ce que Podman en fait (format d'image, timers systemd).
- Appliquer les durcissements de base : non-root, lecture seule, capabilities, `no-new-privileges`.
- Choisir une stratégie de tags, pousser dans un registry privé, nettoyer sans casser, démarrer au boot avec Quadlet.

---

1. Observer

Un conteneur qui tourne produit quatre sources d'information, et chacune a sa commande :

Question	Commande	Ce qu'on y lit
Que dit l'application ?	`podman logs -t --since 10m --tail 50 api`	sa sortie standard et d'erreur, horodatée
Comment a-t-il été lancé ?	`podman inspect api`	image, commande, variables, montages, limites, état, santé
Que fait-il maintenant ?	`podman top api`, `podman stats --no-stream`	processus, CPU, mémoire **réellement** consommés
Que s'est-il passé ?	`podman events --since 1h --filter container=api`	création, démarrage, arrêt, `health_status`, OOM, suppression

Les logs sont la source principale, et la règle du labo 06 s'applique : l'application écrit sur **stdout/stderr**, le moteur capture. Podman les range par défaut dans un fichier par conteneur (pilote `k8s-file`, chemin visible dans `inspect` sous `LogConfig.Path`) ; Docker utilise `json-file`. Dans les deux cas, un fichier qui grossit sans limite finit par remplir le disque : `--log-opt max-size=10m` (Docker) ou `log_size_max` dans `containers.conf` (Podman) le bornent. En entreprise, un agent (Fluent Bit, Promtail, Filebeat) lit ces fichiers et les expédie vers une plateforme centrale (Loki, Elasticsearch) — le conteneur, lui, n'en sait rien.

> **PODMAN**
> Sans daemon, il n'y a pas de flux d'événements central : `podman events` relit un journal écrit par chaque commande (`events_logger` : `journald` sur une Ubuntu avec systemd, `file` sinon). Et `podman stats` exige la **délégation cgroup v2** : sur WSL avec `systemd=true`, c'est le cas ; sans systemd, Podman répond `Error: unable to get cgroup stats` ou affiche des valeurs agrégées absurdes.

2. Limiter

Sans limite, un conteneur peut consommer toute la mémoire de la machine et entraîner les autres dans sa chute. `--memory 256m` plafonne la mémoire, `--cpus 0.5` limite à la moitié d'un cœur, `--pids-limit 200` interdit la bombe à fourches. Les trois se lisent dans `inspect` (`HostConfig.Memory`, `NanoCpus`, `PidsLimit`) et s'appliquent par les cgroups.

> **→ LINUX**
> Les **cgroups v2** organisent les processus en arbre ; chaque nœud porte des fichiers de contrôle : `memory.max` (plafond), `memory.current` (usage), `cpu.max` (quota/période), `pids.max`. Le conteneur est un nœud de cet arbre, et `cat /sys/fs/cgroup/memory.max` **depuis le conteneur** montre sa propre limite. Quand `memory.current` atteint `memory.max` et qu'il n'y a plus rien à libérer, le noyau déclenche l'**OOM killer** : il tue le processus le plus gourmand du groupe, avec `SIGKILL`. Le conteneur sort en `137`, et `inspect` dit `OOMKilled: true`.

> **→ JAVA**
> Depuis Java 10, la JVM lit les cgroups : elle voit `256m`, pas les 32 Go de la machine, et dimensionne son tas en conséquence. Par défaut le tas prend **25 %** de la mémoire du conteneur (la moitié quand la limite est petite, sous 512 Mo) ; `-XX:MaxRAMPercentage=75` le monte à 75 %, en laissant le reste au *metaspace*, aux threads et au code natif. Passé par `JAVA_TOOL_OPTIONS` (labo 08), ce réglage ne touche pas l'image. Une JVM dont le tas dépasse la limite ne lève pas `OutOfMemoryError` : elle est tuée par le noyau. Les deux chiffres, limite du conteneur et pourcentage du tas, se décident ensemble.

3. Surveiller : le `HEALTHCHECK`

Le labo 09 a défini les healthchecks dans Compose. On peut aussi les graver dans l'image : `HEALTHCHECK --interval=5s --timeout=3s --retries=3 CMD wget -qO- http://localhost:8080/actuator/health || exit 1`. Tout conteneur lancé depuis cette image porte alors un état `healthy` / `unhealthy` dans `podman ps`, sans que le déployeur ait à y penser — et Compose ou Kubernetes peuvent le surcharger.

> **PODMAN**
> Deux pièges propres à Podman. Premier : Buildah produit par défaut des images au format **OCI**, qui n'a pas de champ `HEALTHCHECK` ; l'instruction est **ignorée**, avec un simple avertissement au build (`HEALTHCHECK is not supported for OCI image format`) facile à rater, et `podman healthcheck run` répond `has no defined healthcheck`. `podman build --format docker` la conserve. Second : en rootless, la vérification périodique est exécutée par un **timer systemd** par conteneur ; sans session systemd, l'état reste `starting`. Docker n'a ni l'un ni l'autre de ces pièges — mais partage le reste : un healthcheck ne **redémarre** jamais le conteneur, il change seulement son état. C'est un orchestrateur (ou un `--restart` combiné à un `--health-on-failure=kill`, propre à Podman) qui agit.

4. Durcir

Un conteneur en production a cinq réglages de sécurité à vérifier, et aucun ne coûte de code :

1. **Non-root dans l'image** : `USER 1000:1000` (labo 05). En rootless, ce `root` serait déjà votre utilisateur ; sur un Docker root, c'est la différence entre une fuite de conteneur et une compromission de la machine.
2. **Système de fichiers en lecture seule** : `--read-only`, avec `--tmpfs /tmp` pour ce qui doit s'écrire. Un attaquant ne peut ni déposer un binaire ni modifier l'application.

3. **Capabilities minimales** : `--cap-drop ALL`, puis `--cap-add` de ce qui manque réellement (rarement quelque chose pour une API).
4. **Pas d'escalade** : `--security-opt no-new-privileges` interdit aux binaires *setuid* de gagner des droits.
5. **Limites** : mémoire, CPU, PID (section 2) — une protection contre le déni de service, interne ou externe.

> **→ SÉCURITÉ**
> Les **capabilities** découpent les pouvoirs de root en une quarantaine de morceaux : `CAP_CHOWN` (changer un propriétaire), `CAP_NET_BIND_SERVICE` (ports < 1024), `CAP_SYS_ADMIN` (presque tout)… Un conteneur reçoit par défaut une dizaine d'entre elles ; `--cap-drop ALL` les retire toutes, et `chown` répond alors `Operation not permitted` même pour root. C'est le principe du **moindre privilège** appliqué au noyau : ce qui n'est pas accordé ne peut pas être exploité.

Ces options se retrouvent telles quelles dans Compose (`read_only`, `cap_drop`, `security_opt`) et dans Kubernetes (`securityContext`). Elles ne remplacent pas le scan d'image (Trivy, Grype) ni les mises à jour de l'image de base : une CVE dans `glibc` se corrige en reconstruisant, pas en durcissant.

5. Distribuer : tags, digests, registry

Un **tag** est un pointeur mobile ; un **digest** est une empreinte immuable (labo 02). La stratégie d'entreprise classique : la CI pousse `api:1.4.2` (version sémantique), `api:1.4` et `api:1` (pointeurs glissants), plus `api:git-a1b2c3d` (traçable au commit) ; `latest` n'est jamais déployé ; et le déploiement en production référence le **digest** (`api@sha256:…`), seule garantie que ce qui tourne est ce qui a été testé. Un tag de base (`postgres:16-alpine`) se fige de la même façon avant une mise en production.

Le registry privé est la plaque tournante : la CI y pousse, les serveurs y tirent, les scanners y lisent. `registry:2` en est la forme minimale ; en entreprise on trouve Harbor, le registry de GitLab, Nexus, Artifactory, ou ceux des clouds — tous parlent la même API `/v2/`. Un registry sert aussi de **cache** des images publiques, pour ne pas dépendre de Docker Hub et de ses limites de téléchargement.

> **PODMAN**
> Podman exige **TLS** par défaut, même pour `localhost` : `podman push localhost:5000/api:1.0` échoue avec `http: server gave HTTP response to HTTPS client`. `--tls-verify=false` l'autorise pour un registry de test ; pour de bon, on déclare le registry `insecure = true` dans `registries.conf`, ou mieux, on lui donne un certificat. Docker a la même exigence, levée par `insecure-registries` dans `daemon.json`.

6. Nettoyer

Un poste ou un agent de CI accumule : images intermédiaires `<none>` (*dangling*), conteneurs arrêtés, volumes orphelins, cache de build. `podman system df` chiffre chaque catégorie et ce qui est récupérable. Ensuite :

Commande	Supprime	Risque
`podman image prune`	les images dangling	faible : rien de nommé
`podman image prune -a`	**toutes** les images sans conteneur	fort : retire `postgres`, `api:1.0`… à retélécharger ou reconstruire
`podman container prune`	tous les conteneurs arrêtés	moyen : un conteneur arrêté « pour plus tard » disparaît
`podman volume prune`	tous les volumes qu'aucun conteneur n'utilise	**très fort** : une base entre deux `up` est « inutilisée » (labo 06)
`podman system prune`	conteneurs arrêtés + réseaux inutilisés + images dangling (+ `-a`, `--volumes`)	cumule les précédents

PIÈGE
`prune` raisonne en « utilisé par un conteneur existant », jamais en « précieux ». Avant toute suppression de volume, lire `podman volume ls --filter dangling=true` et supprimer **par nom**. Sur un poste, `podman image prune` et `podman container prune` suffisent presque toujours ; `--volumes` et `-a` se réservent à une machine qu'on accepte de voir vide.

7. Démarrer au boot : Quadlet

Le labo 03 l'a montré : sans daemon, `--restart` ne survit pas à `wsl --shutdown`. La réponse de Podman est **Quadlet** : un fichier `~/.config/containers/systemd/api.container`, écrit comme une unité systemd avec une section `[Container]` (`Image=`, `PublishPort=`, `Environment=`, `Volume=`…), que `systemctl --user daemon-reload` transforme en service. `systemctl --user start api` le lance, `enable` l'active au démarrage, `journalctl --user -u api` en montre les logs. Les mêmes fichiers existent pour les réseaux, volumes, pods et fichiers Kubernetes (`.kube`).

→ WINDOWS / WSL
Un service `--user` ne démarre que si votre session utilisateur existe. Pour qu'elle existe sans connexion : `loginctl enable-linger $USER`. Et WSL ne démarre la distribution qu'à l'ouverture d'un terminal (ou via une tâche planifiée Windows) : « au boot » signifie « au premier `wsl` de la journée ». Pour un vrai serveur toujours allumé, Quadlet sur une Linux native ou un cluster Kubernetes.

8. En entreprise : la place dans une chaîne CI/CD

Tout ce que vous avez vu se range dans un *pipeline* : (1) `podman build` d'une image **une seule fois**, avec `--format docker` si un `HEALTHCHECK` est attendu ; (2) **scan** de vulnérabilités, qui bloque sur les CVE critiques ; (3) `push` vers le registry privé avec les tags de version et de commit ; (4) déploiement en recette **par digest**, tests d'intégration via Compose ; (5) promotion du **même digest** en production, par l'orchestrateur. Les limites, le durcissement et le healthcheck sont écrits dans les manifestes, pas tapés à la main. Et l'exploitation du quotidien — logs centralisés, métriques, alertes — repose sur ce que le conteneur expose : `stdout`, `/actuator/health`, `/actuator/prometheus`.

À retenir

- `logs`, `inspect`, `top` / `stats`, `events` : quatre questions, quatre commandes ; les logs vont sur stdout et sont centralisés par un agent.
- `--memory` / `--cpus` / `--pids-limit` passent par les cgroups ; dépassement mémoire = OOM killer, code `137`, `OOMKilled: true` ; la JVM dimensionne son tas sur la limite (`MaxRAMPercentage`).
- `HEALTHCHECK` dans l'image exige `--format docker` avec Podman, et des timers systemd en rootless ; il signale, il ne redémarre pas.
- Durcir : `USER`, `--read-only` + `--tmpfs`, `--cap-drop ALL`, `no-new-privileges`, limites — et scanner l'image.
- Tags de version + de commit, jamais `latest` en production, déploiement par digest ; registry privé avec TLS (`--tls-verify=false` seulement pour le test).
- `prune` ne connaît pas « précieux » : `system df` d'abord, suppression par nom pour les volumes.
- Quadlet remplace `--restart` pour le démarrage au boot ; sur WSL, `enable-linger` et un `wsl` lancé.

Vocabulaire

pilote de logs : `k8s-file` (Podman) / `json-file` (Docker), un fichier par conteneur. — **cgroups v2** : arbre de contrôle des ressources (`memory.max`, `cpu.max`, `pids.max`). — **OOM killer** : mécanisme du noyau qui tue un processus quand la mémoire est épuisée. — **`MaxRAMPercentage`** : part de la mémoire du conteneur allouée au tas Java. — **`HEALTHCHECK`** : commande de santé gravée dans l'image (format docker). — **capabilities** : découpage des privilèges de root. — **`no-new-privileges`** : interdit l'escalade par setuid. — **tag glissant / digest** : pointeur mobile / empreinte immuable. — **registry privé** : dépôt d'images de l'entreprise, aussi cache des images publiques. — **dangling** : image sans tag ni conteneur. — **Quadlet** : fichiers `.container` / `.pod` / `.kube` transformés en unités systemd. — **linger** : session utilisateur systemd maintenue sans connexion.